Anteater Poker Game (Version 1.0)

Software Specification

By All In

Members: Sanaa Bebal, Ava Chinn, Alonso De La Cruz, Estrella Distancia Zamora, Lauren Howe, Samuel Kim, and David Lee

Created for the class EECS 22L at University of California, Irvine

Table of Contents

Glossary

(of terms used in the implementation)

Game

- **Anteater Poker**–a version of poker based on Texas Hold'em, with the addition of the anteater cards and their rules (see "anteater card")
- **Anteater card**–two exist in a deck; not a part of any standard suit; only a ranked hand when a player holds two anteaters in the hole cards, in which case it ranks between a royal and straight flush and four of a kind
- **Board**–a term used to refer to the visible community cards
- **Blinds**–the bids made by the player one left (the small blind) and two left (the big blind) of the dealer; small blind is 1 point and big blind is 2 points
- **Community cards**–the cards visible to all players that can be used by all players to create a five-card hand
- **Dealer**–the player in charge of giving cards to players; dealer deals clockwise
- **Five-card hand**–a group of five cards comprised of some combination of the two hole cards and the five community cards
- **Hand**–refers to the complete sequence of moves made within the repeatable segment of the game, starting with players receiving their cards and ending with someone winning the hand
- **Hole cards**–the two cards held by the individual player; only the player knows what their cards are and only the player can use them to create a five-card hand
- **Kicker**–refers to the cards that are not part of the ranked part of a hand that are used to break ties; used for ties involving one pair, two pair, three of a kind, and four of a kind
- **Pot**–holds the main points bet in a hand
- **Round**–consists of dealing cards and multiple betting intervals where the players match the highest bet, raise, or fold. Rounds include pre-flop, flop, turn, and river betting.
- **Side pot**–not the main pot, but a pot used when at least one player goes all in and the other players continue betting; cannot be won by the player(s) who went all in
- **Suit**–a group of thirteen cards with values ranging from two to ace; the four types ranked lowest to highest are clubs, diamonds, hearts, and spades
- **Value**–refers to the second attribute in addition to suit that a card has; the fourteen values are 2-10, Jack, Queen, King, Ace, and Anteater
- **Game end**–game automatically ends when one player wins all the points or when all players agree to end the game via vote (computer players get no vote)

Betting Stages in a Hand (In Chronological Order)

1. **Preflop**–no cards on the board; betting starts with the small blind and the big blind, then proceeds to the player left of the big blind; each player can call, raise, or fold until all players have either met the highest bet made or folded
    a. Note: During this stage, the only player who can check is the player who made the big blind, and this only occurs when everyone else has called or folded (i.e. not raised); the big blind player can then choose to either check (ending the preflop round) or raise (continuing the round, as all players then have to call, raise, or fold)
2. **Flop**–three cards on the board; each player can check (if no one else has bet), bet, call, raise, or fold until all players have either met the highest bet made or folded
3. **Turn** - After the flop, the dealer reveals the fourth community card; often where draws are complete and the pot size increases dramatically to force decisions
4. **River** - The last community cards is dealt in a game and where the final betting round begins
5. **Showdown** - the final stage of a hand in which all betting has concluded and only occurs when there are at least two players remaining after the final betting round or checking on the river. Typically, the last player to make a bet or raise on the final round is the first to reveal their hand, but if someone checks, the player to the left of the dealer reveals their hand first.

Note: If at any point all but one player has folded in the rounds leading up to the showdown, the one player left wins the pot.

Hand Rankings

1. **Royal flush**–the player has the 10, Jack, Queen, King, and Ace of all one suit
2. **Straight flush**–the player has five cards of consecutive value all of the same suit (i.e. a straight all in the same suit)
3. **Anteater Pair**–the player holds two anteaters in the hole cards
4. **Four of a kind**–the player has four cards of the same rank and one of another rank
5. **Full house** - the hand has three cards of one rank and two cards of another rank
6. **Flush** - a hand contains five cards of sequential order, not necessarily in consecutive order
7. **Straight** - a five-card hand containing sequential ranks, not considering the suits
8. **Three of a kind** - consisting of three cards of the same rank with two unmatched cards
9. **Two pair** - contains two cards of one rank, two cards another rank, and one card of a third rank
10. **Pair** - consists of two cards of the rank and three unrelated or unmatched cards
11. **High card** - contains none of the hands above, the hand is determined by the highest ranking card

The royal flush is the highest-ranking hand possible in the game

Moves

- **All-in**– Wager all of a player's remaining points on a single hand. The player can't take any more betting actions unless they win additional points in any side pots.
- **Bet**– The first voluntary wage made in a betting round.
- **Call**– Match the current highest bet made by the previous player. Only available when a preceding player has made a bet or raise during a betting round.
- **Check**– Pass the action to the next player without wagering. Only available if no player has bet before you in that round.
- **Fold**– Discarding your current hand and forfeit any points contributed in the pot
- **Raise**– Increase the current highest bet, requiring other players to match the new amount to keep their hand. Only available after a bet has been made.

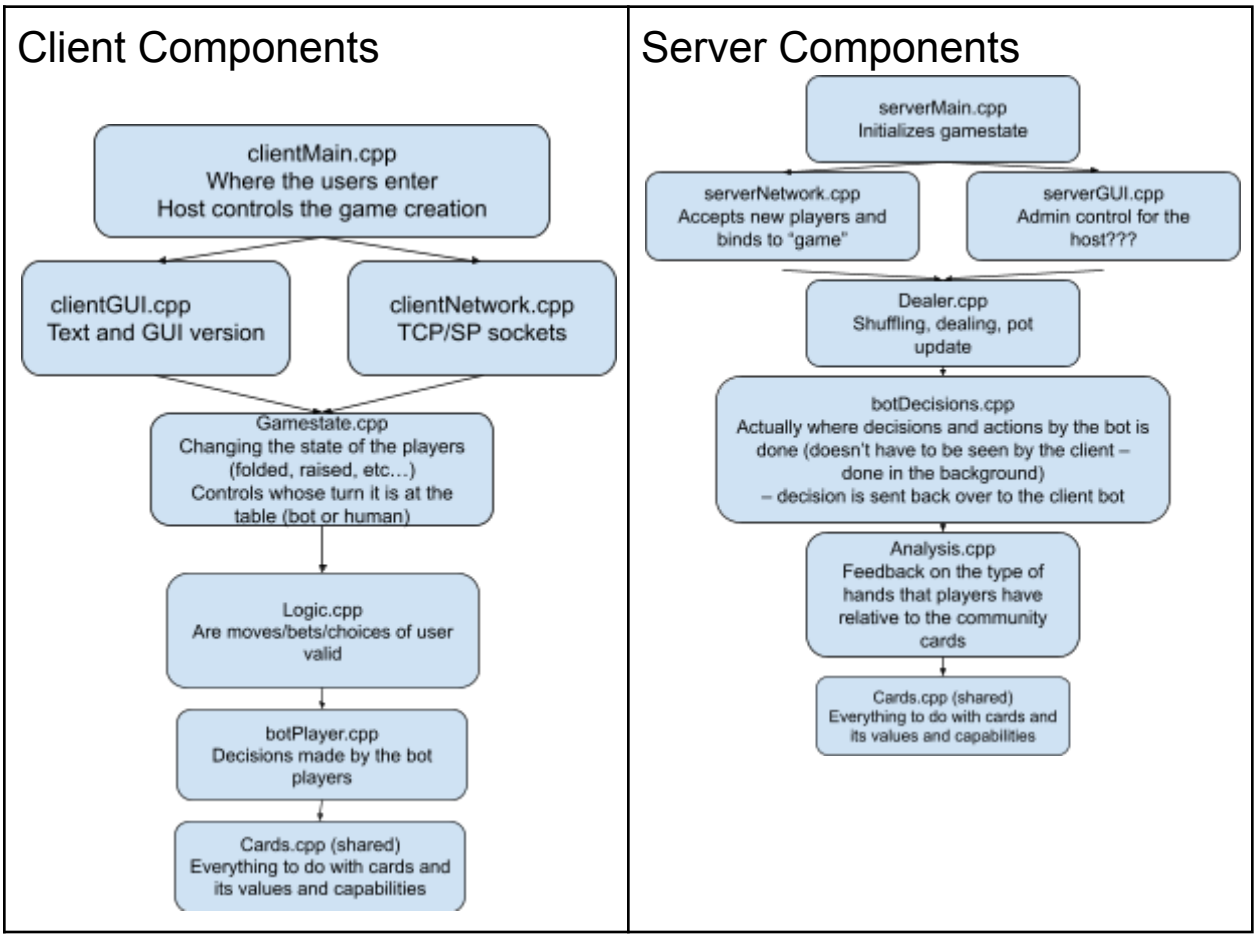
1. Software Architecture Overview

1.1 Main Data Types and Structures

- Enumerators:
    - enum suit–represents Hearts, Clubs, Diamonds, Spades, and Anteaters
    - enum val–represents Two-Ten, Jack, Queen, King, Ace, and Anteater
    - enum Round–represents Preflop, Flop, Turn, River, and Showdown
    - enum PlayerType–represent Human (1) and Computer (2) (players)
    - enum SpecialBets–assigning Check to 0 and Fold to -1
    - enum PackageType–delineates between the types of data sent between the client and server (login and game)
    - Note: Although not an enum, player numbering starts at 0
- Structures:
    - struct gameState
        - int packageType = game // used for sending data via the buffer between the client and the server (see PackageType enum)
        - int numPlayers // holds the number of players in the game
        - int round // holds the round number (Preflop, Flop, Turn, River, or Showdown)
        - int dealerPlayer // holds which player is the "dealer" (note that dealing is done by the server, not the player)
        - PILES allCards // holds all the player hands, the community cards, and the leftover cards (in that order)
        - int pot // represents the amount of money in the pot

        - int callAmount // the amount of money needed to call in the round
        - PLAYERS players // holds player information
        - int playerTurn // holds which player's turn it is
    - struct Player
        - int playerNum // holds player number
        - char *name // holds the player's name
        - int playerType // holds if player is Human (1) or Computer (2) (see PlayerType enum)
        - int score // holds a player's current score (does not include points that have been bet during the hand)
        - int bet // holds a player's bet for the current round of betting
        - int isInHand // still playing in the hand or not (i.e. 1 if they haven't folded, 0 if they have)
        - int isEliminated // should be removed from the list if this is true
        - int playerSocket ??? // represents the client's socket
    - Struct LoginInfo
        - int packageType = login // used for sending data via the buffer between the client and the server (see PackageType enum)
        - int isHost // represents if the player is the "host" player (the host determines the number of players to enter the game, the number of bots, etc.)
        - char *playerName // holds the player's name
        - int playerNum // represents the player number at the table (numbering starts at zero)
        - char *password // set by the player if the host (from client to server), or entered by a joining player if not the host (from server to client)
        - int playerType (Human or Computer (see PlayerType enum))
        - int numPlayers // number of players in the room
        - PLAYERFOUND playersFound // 0 if player is not found; 1 or 2 if player is Human / Computer (see PlayerType enum)
    - typedef struct LoginInfo LOGININFO;
    - typedef struct playerState GAMESTATE;
    - typedef struct Player PLAYER;
- Lists (vectors):
    - typedef std::vector<PLAYER> PLAYERS; // used to hold info about the players
    - typedef std::vector<Card> PILE; // used to hold one of these items: deck, player hands, community cards, leftover cards, etc.
    - typedef std::vector<PILE> PILES; // used to hold player hands, community cards, and leftover cards (in that order)
- Classes:
    - Card class
        - Attributes: val, suit
        - Methods:
            - Card() and Card(int valVar, int suitVar) [constructors]
            - getValue() and getSuit() [getters]

1.2 Major Software Components



1.3 Module Interfaces

Major functions in each module

clientMain.cpp
Local files required: GUI.hpp, Client.hpp, Gamestate.hpp, Logic.hpp, botPlayer.hpp, Analysis.hpp, Cards.hh
Libraries required: <stdio.h>, <stdlib.h>, <string.h>
Internal Functions:

- int main(void)
    - Calls:
- gameLoop()
- closeGame()

- parsingArguments()

clientGUI.cpp
Local files required: gameState.hpp, analysisClient.hpp
Libraries required:

Internal Functions:
- LOGININFO textMenu(void)
    - Calls:
- LOGININFO guiMenu(void)
    - Calls:
- void displayCards(PILES piles)
    - Calls:
- void displayPot(int pot)
    - Calls:
- void displayScores(PLAYERS players)
    - Calls:
- int askAction(PLAYERS players, int playerNum)
    - Calls:
- int displayAnalysis(PILES piles, int playerNum)
    - Calls:
- void displayEndGame(PLAYERS players)
    - Calls:

clientNetwork.cpp
Local files required:
Libraries required: <stdio.h>, <stdlib.h>, <unistd.h>, <string.h>, <sys/types.h>, <sys/socket.h>, <netinet/in.h>, <netdb.h>
Internal Functions:
- void error(const char *msg)
    - Calls: N/A
- int main(int argc, char *argv[])
    - Calls: socket(), gethostbyname(), connect(), bzero(), read()

Gamestate.cpp (Controls everything that pertains to game flow order)
Local files required: Logic.hpp, Gamestate.hpp, Cards.hpp, Analysis.hpp
Libraries required:<stdio.h>, <stdlib.h>, <vector>
Internal Functions:
- GAMESTATE initGamestate(int playerNum)
    - Calls: initDeck()
- int nextPlayer(GAMESTATE *state)
    - Calls: none

- int nextRound(GAMESTATE *state)
    - Calls: none
- int updatePot(GAMESTATE *state, int amount)
    - Calls: none
- int resetHand(GAMESTATE *state)
    - Calls: shuffleDeck(), dealCards()

Logic.cpp (I/O error handling, reactionary functions to player input)
Local files required: Logic.hpp, Gamestate.hpp
Libraries required:<stdio.h>, <stdlib.h>
Internal Functions:

- int validBet(GAMESTATE state, int playerNum, int amount)
    - Calls: none
- int validCall(GAMESTATE state, int playerNum)
    - Calls: none
- int validCheck(GAMESTATE state, int playerNum)
    - Calls: none
- int validRaise(GAMESTATE state, int playerNum, int amount)
    - Calls: validBet()
- int validFold(GAMESTATE state, int playerNum)
    - Calls: none
- int betting(GAMESTATE *state)
    - Calls: validBet(), validCall(), validCheck(), validRaise(), validFold()

botPlayer.cpp (Decisions made by the bot player)
Local files required: Gamestate.hpp, Logic.hpp
Libraries required: <stdio.h>, <stdlib.h>
Internal functions:

serverMain.cpp (initializes gamestate)
Local files required: serverNetwork.hpp, serverGUI.hpp, Dealer.hpp, botDecisions.hpp, Analysis.hpp, Cards.hpp
Libraries required: <stdio.h>, <stdlib.h>, <unistd.h>, <string.h>, <sys/types.h>, <sys/socket.h>, <netinet/in.h>, <netdb.h>
Internal Functions:

- void error()
    - Calls: N/A
- Int main(int argc, char *argv[])
    - Calls: socket(), bind(), listen(), accept(), write()

Cards.cpp (Reshuffles, reinitializes cards in deck; shared between client and server)
Local files required: Cards.hpp, constants.hpp
Libraries required: <vector>, <algorithm>, <crime>, <cstdlib>

Internal Functions:

- PILE initDeck(void)
    - Calls: N/A
- PILE shuffleDeck(PILE deck)
    - Calls: can use srand() from cstdlib
- PILE drawCard(PILE *deck)
    - Calls: N/A
- PILEdealCards(PILE *deck, int playerNum)
    - Calls: drawCard()
- void printCard(Card card)
    - Calls: N/A

serverNetwork.cpp (accepts new players and binds to “game”)
Local files required:
Libraries required: <stdio.h>, <stdlib.h>, <unistd.h>, <string.h>, <sys/types.h>, <sys/socket.h>, <netinet/in.h>, <netdb.h>
Internal functions:

- void error(const char *msg)
    - Calls: N/A
- int main(int argc, char *argv[])
    - Calls: socket(), bzero(), htons(), bind(), listen(), accept(), read()

serverGUI.cpp (...)
Local files required:
Libraries required:
Internal functions:

Dealer.cpp (shuffling, dealing, pot update)
Local files required:
Libraries required: <stdio.h>, <stdlib.h>
Internal functions:

- shuffling()
    - Calls:
- dealing()
    - Calls:
- updatePot()
    - Calls:
- updateCommunityCards()
    - Calls:
- round()
    - Calls:

botDecisions.cpp (where decisions and actions by the bot is done)
Local files required:
Libraries required: <stdio.h>, <stdlib.h>
Internal functions:

- int decideAction(GAMESTATE gameState, int playerNUM)
- evaluateHand()
- int calculatedOdds(int handScore)
- sendToClient()

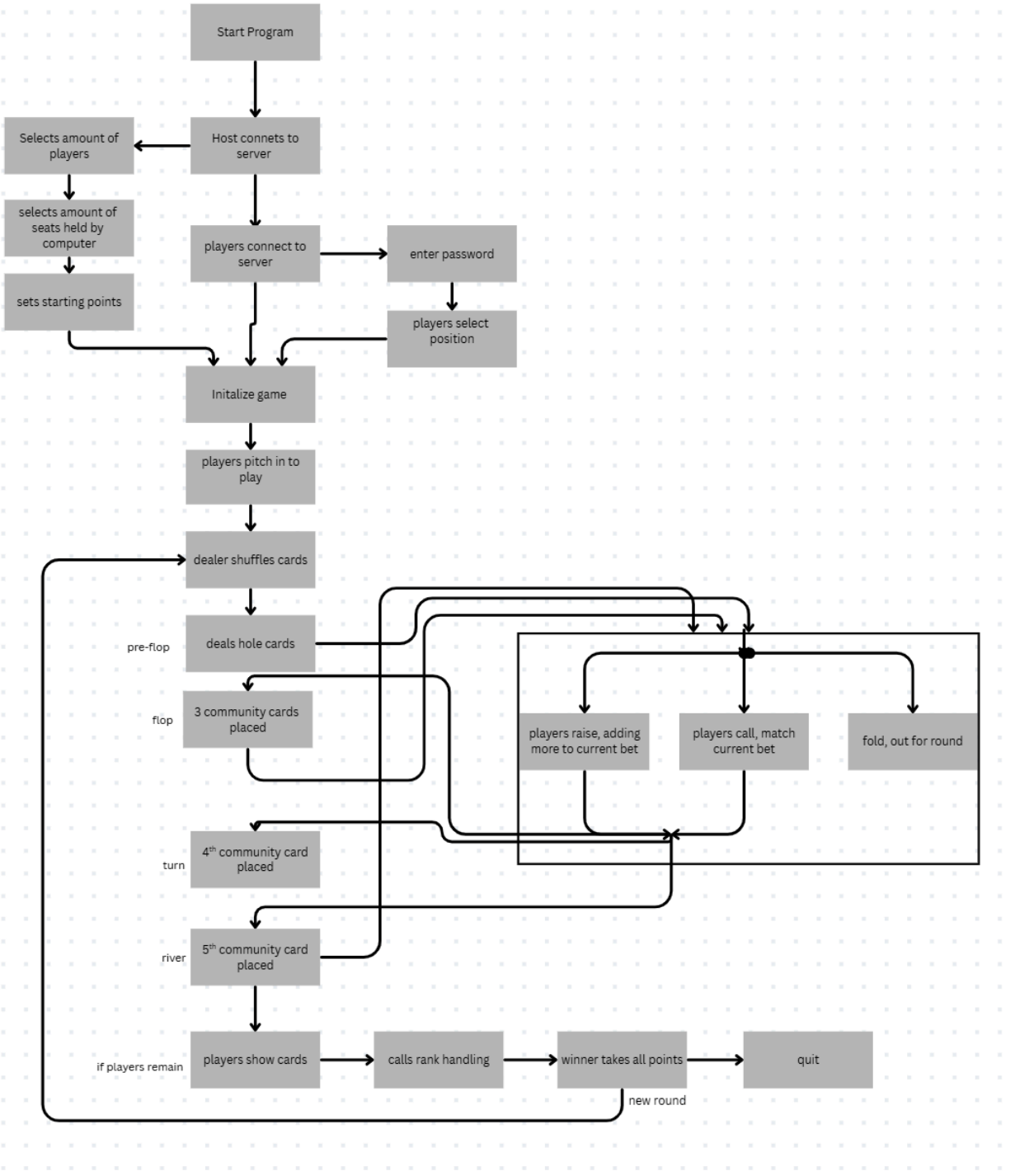
Analysis.cpp (Analyzes each active player's hands)
Local files required:
Libraries required: <stdio.h>, <stdlib.h>
Internal Functions:

- int handHierarchy(PILES piles, int playerNum)
    - Functionality: returns the highest hand based on the following binary functions
    - Calls:
- int royalflush(PILES piles, int playerNum)
    - Functionality: returns 11000 if possible and 0 if not
    - Calls:
- int straightflush(PILES piles, int playerNum)
    - Functionality: returns 1 if possible and 0 if not
    - Calls:
- int antpair(PILES piles, int playerNum)
    - Functionality: return 1 if possible and 0 if not
    - Calls:
- int ofakind(PILES piles, int playerNum)
    - Functionality: Returns an int in (1000, 2000] for a pair, in (3000,4000] for 3-of-a-kind, and in (7000,8000] for 4-of-a-kind
    - Calls:
- int fullhouse(PILES piles, int playerNum)
    - Functionality: returns an int in (6000,7000]
    - Calls:
- int flush(PILES piles, int playerNum)
    - Functionality: returns an int in (5000,6000]
    - Calls:
- int straight(PILES piles, int playerNum)
    - Functionality: returns an int in (4000,5000]
    - Calls:
- int twopair(PILES piles, int playerNum)
    - Functionality: returns an int in (2000,30000]
    - Calls:

- int highcard(PILES piles, int playerNum)
    - Functionality: returns an int within (0,1000]
    - Calls:

Deck.cpp (deals with card selection & initialization)
Local files required:
Libraries required: <stdio.h>, <stdlib.h>
Internal functions:

1.4 Overall Program Control Flow



1.5 Automated Client: Poker Bot

2. Installation

2.1 System requirements and compatibility

The user must satisfy the following conditions to correctly install the software:

- **Operating System:** RHEL8 64-bit Linux
- **Compiler:** GNU Compiler Collection (gcc)
- **Build System:** GNU *make*
- **Libraries:** Standard C libraries; GTK 3.0
- **Utilities:** tar, gzip, evince, and shell environment
- **Hardware Requirements:** Minimal CPU and memory resources sufficient for compiling and executing.
- **Server Access**: Must be able to access the UCI EECS Server

2.2 Setup and configuration

Using the command line, go into the installed folder and run the following code:

A. **Extract the Source Package**
   gtar xvzf Poker_V1.0_src.tar.gz
   cd poker
B. **Build the Software**
   make *(from within the poker directory)*
C. **Run the Executable**
   bin/poker *(from within the poker directory)*
D. **Remove compiled object files**
   make clean *(from within the poker directory)*

2.3 Building, compilation, and installation

A. **Create the Binary Distribution**
   tar -zcvf Poker_V1.0.tar.gz \
   ./doc/README \
   ./doc/COPYRIGHT \
   ./doc/Poker_UserManual.pdf \
   ./doc/INSTALL \
   ./bin/poker \
   ./bin/*.bmp \
   ./Makefile
B. **Create the Source Distribution**
   tar -zcvf Poker_V1.0_src.tar.gz \
   ./doc \

./bin \
./src \
./Makefile

C. Purpose of Each Package

Poker_V1.0.tar.gz - Contains the executable and documentation for end users
Poker_V1.0_src.tar.gz - Contains full source code for developers and maintainers

3. Documentation of Packages, Modules, and Interfaces

3.1 Detailed Description of Data Structures

Card class (cards.hpp):

```
class Card{
public:
    int val;
    int suit;

    Card(void){ // overloading
        val = -1;
        suit = -1;
    }
    Card(int valVar, int suitVar){
        val = valVar;
        suit = suitVar;
    }

    int getValue(void) const { // const means funct can't modify
anything--good for all getters
        return val;
    }
    int getSuit(void) const{
        return suit;
    }

};
```

3.2 Detailed Description of Functions and Parameters

clientMain.cpp -
Provides: where main game flow is run and where all functions are called upon
Requires:

int main(void)
- Arguments:
- Result:

gameLoop()
- Arguments:
- Result:

closeGame()
- Arguments:
- Result:

parsingArguments()
- Arguments:
- Result:

clientGUI.cpp -
Provides: text and GUI application
Requires:

LOGININFO textMenu(void)
- Arguments:
- Result: returns a structure w/ login info

LOGININFO guiMenu(void)
- Arguments:
- Result: returns a structure w/ login info

void displayCards(PILES piles)
- Arguments: vector containing hand/board/extra card vectors
- Result:

void displayPot(int pot)
- Arguments:
- Result: displays the pot

void displayScores(PLAYERS players)
- Arguments: vector containing hand/board/extra card vectors
- Result:

int askAction(PLAYERS players, int playerNum)
- Arguments: vector containing hand/board/extra card vectors
- Result:

int displayAnalysis(PILES piles, int playerNum)

- Arguments: vector containing hand/board/extra card vectors
- Result:

void displayEndGame(PILES piles, int playerNum)

- Arguments: vector containing hand/board/extra card vectors
- Result:

clientNetwork.cpp -
Provides:
Requires:

void error(void)

- Arguments: called when system call fails
- Result: returns error statement

Int main(void)

- Arguments:
- Result:

gameState.cpp -
Provides: controls the next actions of the game
Requires:

Int checkValid(GAMESTATE gameState, int proposedMove, int playerNum)

- Arguments:
- Result: return int 1 if valid and 0 if not

GAMESTATE nextPlayer(GAMESTATE gameState)

- Arguments: passes turn to next player still in the game
- Result:

Logic.cpp -
Provides: checks whether moves/bets/choices of users are valid
Requires:

int validBet(GAMESTATE gameState, int playerNum, int amount)

- Arguments: checks to see if player entered a valid bet
- Result: returns int 1 if a valid bet (i.e. no one else has made a bet so far in the round) and 0 if not

int validCheck(GAMESTATE gameState, int playerNum)

- Arguments: determines if check is available to player
- Result: returns 1 if check is valid (i.e. no one else has made a bet so far in the round) and 0 if not

int validAllIn(GAMESTATE gameState, int playerNum, int amount)

- Arguments: determines if all in is available to player

- Result: returns 1 if all in is valid (i.e. player has enough points and it is all of their remaining points) and 0 if not

int validFold(GAMESTATE gameState, int playeNum, int amount)

- Arguments: determines if fold is available to player
- Result: returns 1 if fold is valid (primarily checking if round isn't showdown) and 0 if not

Int validCall(GAMESTATE gameState, int playerNum, int amount)

- Arguments: determines if call is available to player
- Result: returns 1 if call is valid (i.e. player has enough points) and 0 if not

Int validRaise(GAMESTATE gameState, int playerNum, int amount)

- Arguments: determines if raise is available to player and player has enough points
- Result: returns 1 if raise is valid (i.e. player has enough points) and 0 if not

Int getValidMoves(GAMESTATE gameState, int playerNum, int amount)

- Arguments: calls functions above to determine what moves are valid
- Result: returns an array with each entry representing a different option

botPlayer.cpp -

Provides: receives information (decisions of the current bot player) from the server and applies to client's screen and application

Requires:

acceptBotDecision()

- Arguments: receives information from serverBot
- Result:

serverMain.cpp-

Provides: initializes the state of the game based upon the specifications of the host user (will provide info such as how many decks are use, how many player, etc.)

Requires:

int main(void)

- Arguments: main function on the server that runs the game
- Result:

GAMESTATE initGameState(LOGININFO loginInfo)

- Arguments:
- Result: initiates the GAMESTATE structure based on the login information

int runGame(GAMESTATE startGameState)

- Arguments:
- Result:

int handlingVotes()

- Arguments: prompted if a player would like to quit then prompts all other players if they would like to end the game early
- Result: returns 1 if players all players are in agreeance of quitting and 0 otherwise

Cards.cpp -
Provides:
Requires:

PILE initDeck(void)
- Arguments:
- Result: returns an initialized deck

FILE one Shuffle(PILE deck)
- Arguments:
- Result: returns a deck shuffled once

PILES deal(PILE deck, int numPlayers)
- Arguments:
- Result: creates a vector list of the player hands and the community cards including leftover cards

serverNetwork.cpp -
Provides:
Requires:

void error(void)
- Arguments:
- Result:

int main()
- Arguments:
- Result:

serverGUI.cpp -
Provides:
Requires:

void placeholder(...)
- Arguments:
- Result:

Dealer.cpp -
Provides:
Requires:

void placeholder(...)
- Arguments:
- Result:

botDecisions.cpp -
Provides: all logic needed for the bot players to make decisions (all settled actions will be sent to the client to be completed)

Requires:

Int decideAction(GAMESTATE gameState, int playerNum)
- Arguments: makes final decision of what move to make
- Result: returns a number that reflects the player's action (bet, call, fold, etc.)

evaluateHand()
- Arguments:
- Result:

Int calculateOdds(int handScore)
- Arguments: determines likelihood of winning based on current hand and flop
- Result:

sendToClient()
- Arguments:
- Result:

analysis.cpp -
Provides:
Requires:

int hand-hierarchy(...)
- Arguments:
- Result:

int royalflush()
- Arguments:
- Result:

int straightflush()
- Arguments:
- Result:

int threeofakind()
- Arguments:
- Result:

int full

Deck.cpp -
Provides:
Requires:

void placeholder(...)
- Arguments:
- Result:

EXAMPLE

main.c - entry point and entering the game loop

Provides: program entry and the top-most level game loop

Requires: user.h, rules.h, chess.h, gamestate.h, computer.h, moves.h

int main(void)

- Arguments: none
- Result:
    - 0 on clean exit

3.3 Detailed Description of Communication Protocol

- Explanation of communication related function calls
- Explanation of communication protocol flags/settings

4. Development Timeline

4.1 Partitioning of Tasks

Week 6	Week 7	Week 8	Week 9	Week 10
Create user manual	Create software specification Divide work amongst team members	Create alpha version of code with text-based interface Focus on relaying messages to clients via the server	Create beta version of code Fix any bugs with alpha version	Create final version of the code

4.2 Team Member Responsibilities

Sanaa Bebal - clientNetwork.cpp / serverNetwork.cpp / clientGUI.cpp / serverGUI.cpp / clientMain.cpp

Ava Chinn - logic.cpp / cards.cpp / botDecisions.cpp / botPlayer.cpp
Alonso De La Cruz - logic.cpp / cards.cpp / botDecisions.cpp / botPlayer.cpp
Estrella Distancia - clientMain.cpp / serverMain.cpp / serverGUI.cpp
Lauren Howe - dealer.cpp / gamestate.cpp / cards.cpp / analysis.cpp
Samuel Kim - logic.cpp / gamestate.cpp / analysis.cpp
David Lee - gamestate.cpp / logic.cpp / analysis.cpp / dealer.cpp

5. References and External Libraries

https://github.com/jmqueen01/EECS22L-chess/blob/main/README
Used as a reference for installation Linux code

Contact Information

If you have any questions, comments, or concerns about this software, please contact the creators using the following email addresses:

- sbebal@uci.edu
- amchinn@uci.edu
- alonsojd@uci.edu
- edistanc@uci.edu
- lehowe@uci.edu
- samuek16@uci.edu
- davijl18@uci.edu

Note: Any email pertaining to this software sent to one or more of the addresses above may be forwarded to any or all of the other emails above.

Terms and Conditions

Copyright

Disclaimer of Warranty

THIS SOFTWARE IS PROVIDED “AS IS” WITHOUT ANY EXPRESS OR IMPLIED WARRANTY OF ANY KIND.

Index